Green University of Bangladesh
Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering
Semester: (Spring, Year: 2026), B.Sc. in CSE (Day)

Lab Report NO: 03
Course Title: AI Lab
Course Code: CSE 316
Section: 232-D5

Lab Experiment Name: Implementation of IDDFS and A* Search Algorithm

Student Details

	Name	ID
1.	Ashab Uddin	232002274

Lab Date : **17/03/2026**
Submission Date : **30/03/2026**
Course Teacher's Name : **Babe Sultana**

Lab Report Status

Marks: …… **Signature:.....**
Comments:..... **Date:.....**

1. INTRODUCTION

Search algorithms form the backbone of problem-solving techniques in Artificial Intelligence, especially when dealing with state-space graphs. This laboratory experiment focuses on the practical implementation of two distinct search strategies: Iterative Deepening Depth-First Search (IDDFS) and the A* (A-Star) Search Algorithm.

IDDFS combines the space efficiency of Depth-First Search with the completeness of Breadth-First Search by repeatedly performing depth-limited searches with an incrementally increasing depth ceiling. This makes it particularly suitable for large or infinite search spaces where memory is a constraint.

A* Search, on the other hand, is an informed search technique that leverages a heuristic function to intelligently estimate the cost from any given node to the goal. By evaluating each node using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual accumulated cost and $h(n)$ is the heuristic estimate, A* can identify the shortest path more efficiently than uninformed approaches.

2. OBJECTIVES/AIM

- To construct and traverse a dynamic graph using the IDDFS strategy
- To locate a path between a given source and destination using IDDFS with backtracking
- To apply the A* algorithm on a predefined weighted graph
- To examine how heuristic estimates, shape the efficiency and direction of the A* search

3. PROCEDURE

IDDFS Graph Traversal

1. Begin with a depth limit set to zero
2. Execute a Depth-Limited Search (DLS) starting from the source node
3. Print any node reachable within the current depth
4. Increment the depth limit by one after each complete pass
5. Repeat until the maximum specified depth is reached

IDDFS Path finding:

1. Maintain a running list representing the active traversal path
2. At every node visited, check whether it matches the goal
3. If a match is found, return the complete path accumulated so far
4. If not, recursively explore neighbors while tracking the path
5. Use backtracking to remove nodes that do not lead toward the goal
6. Continue deepening the search until the path is successfully discovered

A* Search

1. Set up a min-heap priority queue and insert the starting node with its heuristic value
2. Pop the node with the lowest f-value from the queue at each iteration
3. Display the state of the priority queue before each pop to trace execution
4. Expand the popped node's neighbors, computing updated g and f values
5. Push valid unexplored neighbors back into the queue
6. Continue until the destination node is extracted from the queue

4. IMPLEMENTATION:

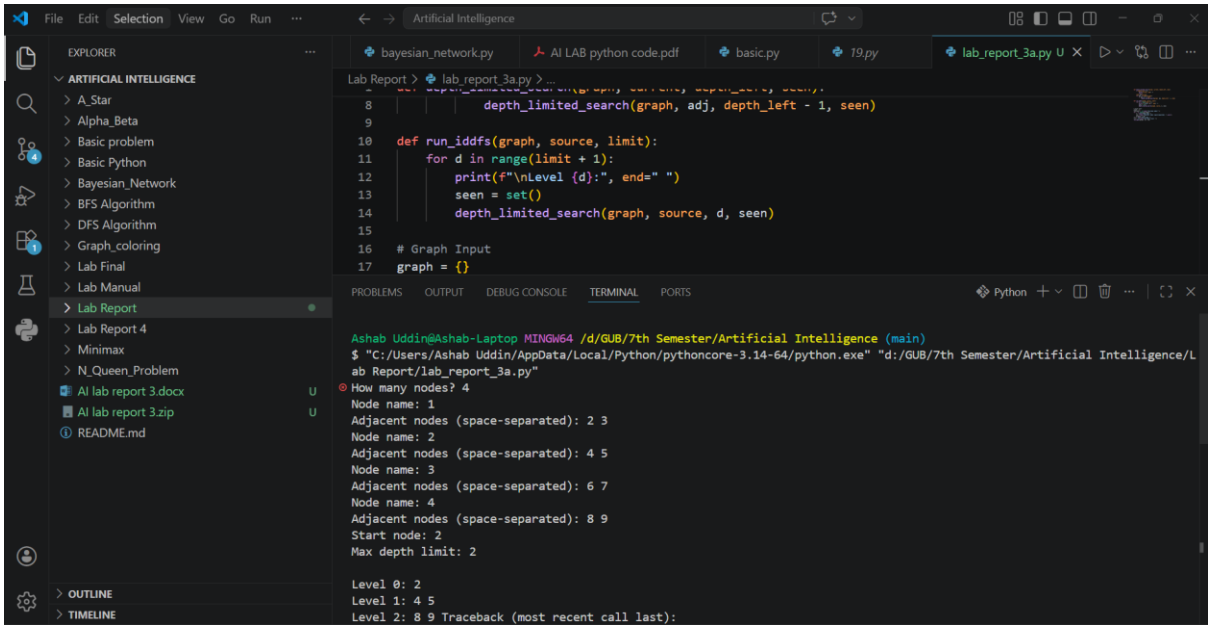
Code:

Problem 1: IDDFS Traversal on a User-Defined Graph

The following program accepts a graph from the user at runtime and performs IDDFS traversal, printing all nodes reachable at each depth level.

```
def depth_limited_search(graph, current, depth_left, seen):
    if depth_left == 0:
        print(current, end=" ")
        return
    seen.add(current)
    for adj in graph[current]:
        if adj not in seen:
            depth_limited_search(graph, adj, depth_left - 1, seen)

def run_iddfs(graph, source, limit):
    for d in range(limit + 1):
        print(f"\nLevel {d}:", end=" ")
        seen = set()
        depth_limited_search(graph, source, d, seen)

# Graph Input
graph = {}
num_nodes = int(input("How many nodes? "))
for _ in range(num_nodes):
    n = input("Node name: ")
    adj = input("Adjacent nodes (space-separated): ").split()
    graph[n] = adj
src = input("Start node: ")
lim = int(input("Max depth limit: "))
run_iddfs(graph, src, lim)
```

Output:



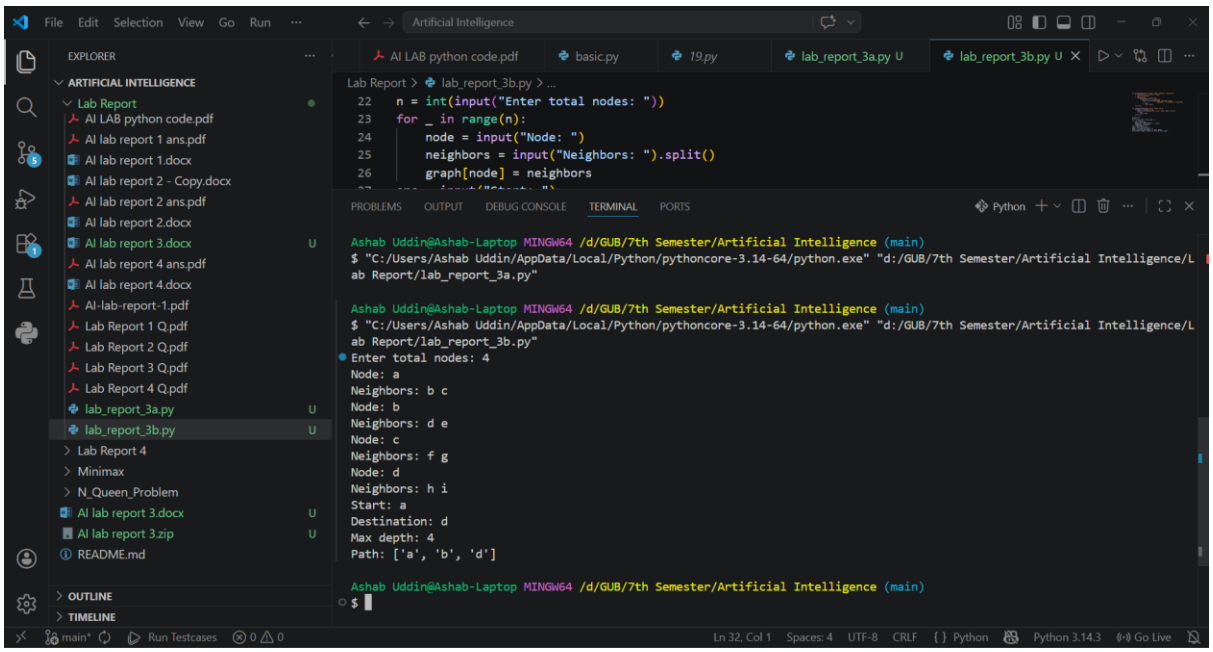
Problem 2: Path Discovery Using IDDFS

This program extends IDDFS to locate and return a path from a specified starting node to a target node. It uses recursive backtracking to explore routes and stops once the path is found.

```
def dls_find_path(graph, current, target, remaining, visited_path):
    if remaining == 0 and current == target:
        return visited_path
    if remaining > 0:
        for next_node in graph[current]:
            if next_node not in visited_path:
                result = dls_find_path(graph, next_node, target,
                                       remaining - 1, visited_path +
[next_node])
                if result:
                    return result
    return None

def find_path_iddfs(graph, source, target, max_d):
    for depth in range(max_d + 1):
        found = dls_find_path(graph, source, target, depth, [source])
        if found:
            return found
    return None

# Graph Input
graph = {}
```

```python
n = int(input("Enter total nodes: "))
for _ in range(n):
    node = input("Node: ")
    neighbors = input("Neighbors: ").split()
    graph[node] = neighbors
src = input("Start: ")
dest = input("Destination: ")
depth = int(input("Max depth: "))
path = find_path_iddfs(graph, src, dest, depth)
print("Path:", path) if path else print("No route found")
```

Output:



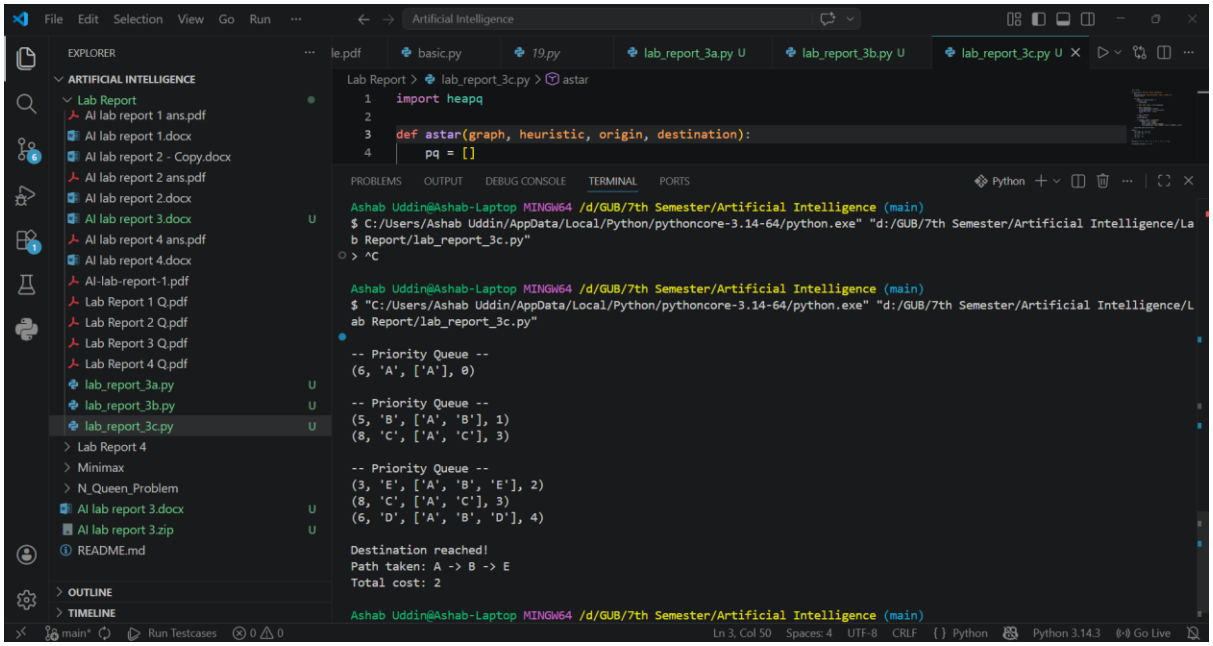
Problem 3: A* Search Algorithm with Queue Visualization

This implementation applies the A* algorithm on a fixed weighted graph. After each step, the current state of the priority queue is printed so the exploration process can be closely observed.

```python
import heapq

def astar(graph, heuristic, origin, destination):
    pq = []
    heapq.heappush(pq, (heuristic[origin], origin, [origin], 0))
    visited = set()

    while pq:
        print("\n-- Priority Queue --")
        for entry in pq:
```

```
            print(entry)

        f, node, route, cost_g = heapq.heappop(pq)

        if node == destination:
            print("\nDestination reached!")
            print("Path taken:", " -> ".join(route))
            print("Total cost:", cost_g)
            return

        if node in visited:
            continue
        visited.add(node)

        for neighbor, weight in graph[node]:
            if neighbor not in visited:
                g_new = cost_g + weight
                f_new = g_new + heuristic[neighbor]
                heapq.heappush(pq, (f_new, neighbor, route + [neighbor],
g_new))

    print("Path could not be found.")

graph = {
    'A': [('B', 1), ('C', 3)],
    'B': [('D', 3), ('E', 1)],
    'C': [('F', 5)],
    'D': [],
    'E': [('F', 2)],
    'F': []
}

heuristic = {'A': 6, 'B': 4, 'C': 5, 'D': 2, 'E': 1, 'F': 0}

astar(graph, heuristic, 'A', 'E')
```

OUTPUTS:



6. ANALYSIS AND DISCUSSION:

Both algorithms were successfully implemented and produced accurate results. IDDFS proved to be a memory-friendly approach since it does not maintain an open list of all visited nodes simultaneously. However, it involves repeated traversal of the same nodes at shallower depths before incrementing the depth limit, which leads to some computational redundancy.

In contrast, the A* algorithm demonstrated superior efficiency for the given graph by using heuristic guidance to prioritize nodes that appear more promising. The h(n) values steered the search away from less optimal routes and toward the goal more directly. Displaying the priority queue at each iteration was particularly insightful, as it revealed how the selection criterion f(n) = g(n) + h(n) progressively narrows the search.

One key challenge encountered during IDDFS implementation was ensuring proper backtracking so that the path list was updated correctly at each recursion level. For the A* implementation, careful handling of the heap structure was necessary to ensure already-visited nodes were not re-expanded. Both issues were resolved through careful debugging and testing with multiple graph configurations.

Overall, this experiment reinforced the conceptual distinction between uninformed and informed search strategies and demonstrated the practical trade-offs between memory usage, speed, and optimality. The objectives of the lab were fully met through hands-on coding and experimentation.